

LAPORAN PROYEK
IMPLEMENTASI TEKNIK SORTING DAN SEARCHING
LANJUTAN

Dosen Pengampu : Ali Akbar Lubis, S.Kom., M.TI.

Mata Kuliah : Struktur Data



Disusun Oleh :

Kelompok 4 PTIK D 2025

Giovani Fernando Perdamean Hutagalung	: 5253151037
Muhammad Fazry Chandra	: 5253151050
Jonatan Marganda Parulian Manullang	: 5253151059

PROGRAM STUDI PENDIDIKAN TIK
FAKULTAS TEKNIK
UNIVERSITAS NEGERI MEDAN

2026

KATA PENGANTAR

Puji dan syukur Penulis panjatkan kepada Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, Penulis dapat menyelesaikan laporan proyek ini dengan judul “Laporan Proyek Implementasi Teknik Sorting dan Searching Lanjutan”. Laporan ini disusun untuk memenuhi tugas yang diberikan oleh dosen pengampu mata kuliah Struktur Data bapak Ali Akbar Lubis, S.Kom., M.TI.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan agar penulis dapat memperbaiki diri di masa mendatang. Ucapan terima kasih penulis sampaikan kepada pihak yang telah berkontribusi dalam penyusunan laporan ini. Penulis juga memohon maaf jika di dalam laporan ini terdapat banyak kesalahan dan kekurangan. Semoga laporan ini dapat memberikan manfaat, dan menambah pengetahuan bagi semua pihak yang membutuhkan serta dapat dijadikan referensi dalam pembelajaran.

Medan, 03 Juni 2026

Kelompok 4

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
DAFTAR TABEL.....	iv
DAFTAR GAMBAR.....	v
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian.....	3
1.4 Manfaat Penelitian.....	3
BAB II LANDASAN TEORI	4
2.1 Algoritma dan Kompleksitas Waktu	4
2.2 Counting Sort	4
2.2.1 Prinsip Kerja Counting Sort.....	5
2.2.2 Visualisasi Counting Sort.....	5
2.2.3 Kompleksitas Counting Sort	6
2.2.4 Keunggulan dan Keterbatasan Counting Sort	6
2.3 Binary Search	7
2.3.1 Prinsip Kerja Binary Search.....	7
2.3.2 Visualisasi Binary Search	8
2.3.3 Kompleksitas Binary Search.....	8
2.4 Perbandingan Kompleksitas Algoritma.....	8
2.5 Pseudocode	9
2.5.1 Psuedocode Counting Sort	9
2.5.2 Pseudocode Binary Search.....	10
BAB III DESAIN DAN IMPLEMENTASI PROGRAM.....	11
3.1 Arsitektur Program	11
3.2 Struktur Data Yang Digunakan	11
3.3 Tampilan Hasil Saat Dieksekusi.....	12
BAB IV PEMBAHASAN.....	13
4.1 Deskripsi Kasus Nyata	13
4.2 Dataset Yang Digunakan.....	13
4.3 Pembahasan Algoritma.....	14
4.3.1 Alasan Counting Sort Lebih Baik dari Bubble Sort	14

4.3.2 Analisis Kestabilan Sorting	15
4.4 Analisis Hasil Eksperimen	15
4.4.1 Analisis Berdasarkan Ukuran Data.....	16
4.4.2 Analisis Berdasarkan Kondisi Data	17
4.4.3 Analisis Performa Binary Search.....	18
BAB V KESIMPULAN DAN SARAN.....	19
5.1 Kesimpulan.....	19
5.2 Saran.....	19
DAFTAR PUSTAKA	21

DAFTAR TABEL

Tabel 2. 1 Perbandingan Kompleksitas Waktu dan Ruang Algoritma	9
Tabel 4. 1 Hasil Benchmark Perbandingan Counting Sort vs Bubble Sort.....	16

DAFTAR GAMBAR

Gambar 2. 1 Visualisasi Array Input Counting Sort	5
Gambar 2. 2 Array Frekuensi (Count Array) Setelah Pencacahan	6
Gambar 2. 3 Visualisasi Binary Search (Mencari nilai 70)	8
Gambar 2. 4 Perbandingan Kompleksitas Waktu Algoritma.....	9
Gambar 3. 1 Arsitektur Modular Program Sorting & Searching.....	11
Gambar 4. 1 Perbandingan Waktu Eksekusi Counting Sort vs Bubble Sort.....	17
Gambar 4. 2 Jumlah Operasi vs Kondisi Data ($n = 1.000$)	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang sangat pesat ini menuntut kemampuan perangkat lunak untuk mengelola dan memproses data dalam jumlah yang semakin besar dan kompleks. Salah satu operasi yang paling fundamental dan sering digunakan dalam ilmu komputer adalah pengurutan (Sorting) dan pencarian (Searching) data. Hampir setiap sistem perangkat lunak modern, mulai dari mesin pencari (*Search Engine*), platform perdagangan elektronik, sistem manajemen basis data, hingga aplikasi analitik data, senantiasa mengandalkan algoritma Sorting dan Searching yang efisien untuk mengelola dan menemukan informasi secara cepat dan akurat.

Tiga algoritma sorting dasar yaitu Bubble Sort, Insertion Sort, dan Selection Sort memiliki kelebihan dalam hal kemudahan pemahaman dan implementasi, namun memiliki kompleksitas waktu $O(n^2)$ yang menjadikannya sangat lambat ketika dihadapkan pada data dalam skala yang besar. Sebagai ilustrasi, pada data sejumlah 10.000 elemen, Bubble Sort dapat memerlukan hingga 100 juta operasi perbandingan dalam kondisi terburuk, suatu jumlah yang tidak dapat diterima dalam lingkungan produksi yang membutuhkan respons cepat.

Untuk mengatasi keterbatasan tersebut, para peneliti telah mengembangkan berbagai algoritma sorting yang lebih efisien. Salah satu pendekatan yang paling menonjol adalah Counting Sort. Menurut Song dan Li (2023), algoritma ini merupakan “*A linear time complexity sorting algorithm that is suitable for cases where the input data has a relatively small range of values*”. Counting Sort adalah algoritma pengurutan berbasis non perbandingan (*Non comparison based*) yang mencapai kompleksitas waktu $O(n + k)$, dimana n adalah jumlah elemen dan k adalah rentang nilai data. Seperti dijelaskan dalam literatur “*Counting Sort is a non comparison based sorting algorithm that sorts integers or objects with discrete keys by counting the occurrences of each unique element*”.

Algoritma ini tidak melakukan perbandingan antar elemen secara langsung, melainkan memanfaatkan teknik pencacahan frekuensi untuk menentukan posisi setiap elemen dalam array terurut. Pendekatan ini terbukti jauh lebih efisien

dibandingkan algoritma berbasis perbandingan untuk kasus data integer dengan rentang nilai yang terbatas, karena “*Its linear time complexity, $O(n + k)$, makes it faster than comparison based algorithms*” ketika k tidak jauh lebih besar daripada n .

Di sisi pencarian, Binary Search merupakan algoritma yang sangat efisien dengan kompleksitas $O(\log n)$. Algoritma ini bekerja dengan cara membagi ruang pencarian menjadi dua bagian secara berulang, sehingga jumlah langkah yang diperlukan untuk menemukan suatu elemen hanya bertumbuh secara logaritmik terhadap ukuran data. Pada data sejumlah 10.000 elemen, Binary Search hanya memerlukan maksimal 14 langkah perbandingan, sedangkan pencarian linear (*Linear Search*) memerlukan hingga 10.000 langkah dalam kondisi terburuk.

Berdasarkan latar belakang yang ada, penulis memilih untuk mengimplementasikan Counting Sort sebagai algoritma sorting dan Binary Search sebagai algoritma searching dalam penelitian ini. Studi kasus yang diangkat adalah pengurutan dan pencarian data nilai ujian mahasiswa, yang merupakan konteks nyata serta sangat relevan dalam dunia pendidikan. Data nilai ujian tersebut berada dalam rentang integer terbatas ($0 - 100$), sehingga sangat sesuai dengan karakteristik efisiensi Counting Sort.

1.2 Rumusan Masalah

1. Bagaimana cara kerja dan mekanisme algoritma Counting Sort dalam melakukan pengurutan data integer?
2. Bagaimana cara kerja dan mekanisme algoritme Binary Search dalam melakukan pencarian data pada array terurut?
3. Bagaimana implementasi kedua algoritma tersebut dalam bahasa pemrograman C++ dengan fitur-fitur program yang komprehensif?
4. Bagaimana perbandingan performa Counting Sort terhadap Bubble Sort ditinjau dari waktu eksekusi dan jumlah operasi pada berbagai ukuran dan kondisi data?
5. Pada kondisi data seperti apa Counting Sort menunjukkan keunggulan tertinggi dan pada saat kapan kurang optimal?

1.3 Tujuan Penelitian

1. Memahami dan menjelaskan cara kerja, pseudocode, serta analisis kompleksitas waktu dan ruang algoritma Counting Sort dan Binary Search secara mendalam.
2. Mengimplementasikan algoritma Counting Sort dan Binary Search dalam bahasa pemrograman C++ dengan fitur input data yang beragam (Manual, acak, dan dari file).
3. Melakukan analisis eksperimental yang membandingkan performa Counting Sort terhadap Bubble Sort menggunakan berbagai ukuran data dan kondisi data.
4. Menerapkan kedua algoritma pada studi kasus pengurutan dan pencarian data nilai ujian mahasiswa sebagai representasi masalah nyata.
5. Menyajikan hasil analisis dalam bentuk tabel perbandingan, grafik, dan pembahasan ilmiah yang komprehensif.

1.4 Manfaat Penelitian

1. Bagi penulis, proyek ini memperdalam pemahaman tentang algoritma sorting dan searching lanjutan, serta melatih kemampuan implementasi dan analisis algoritmik secara empiris.
2. Bagi pembaca, laporan proyek ini dapat dijadikan sebagai referensi dalam pembelajaran mata kuliah Struktur Data, khususnya pada topik algoritma sorting dan searching lanjutan.
3. Bagi pengembang perangkat lunak, analisis komparatif yang disajikan memberikan panduan praktis dalam memilih algoritma yang tepat sesuai karakteristik data yang dihadapi.
4. Bagi ilmu pengetahuan, proyek ini memberikan bukti empiris mengenai keunggulan dan keterbatasan Counting Sort dibandingkan algoritma sorting berbasis perbandingan.

BAB II LANDASAN TEORI

2.1 Algoritma dan Kompleksitas Waktu

Algoritma merupakan sekumpulan langkah-langkah terdefinisi, terurut, dan terbatas yang digunakan untuk menyelesaikan suatu masalah atau tugas komputasi (Cormen et al., 2022). Setiap algoritma memiliki karakteristik efisiensi yang dapat diukur menggunakan analisis kompleksitas, yaitu pengukuran sumber daya komputasi (Waktu dan memori) yang digunakan algoritma sebagai fungsi dari ukuran inputnya.

Notasi Big – O (O) merupakan notasi asimtotik yang paling umum digunakan untuk mengekspresikan kompleksitas waktu terburuk (*Worst case time complexity*) suatu algoritma. Notasi ini menggambarkan batas atas pertumbuhan fungsi waktu algoritma ketika ukuran input mendekati tak hingga. Sebagai contoh, $O(n^2)$ berarti waktu eksekusi algoritma tumbuh secara kuadratik terhadap ukuran input n , sedangkan $O(n \log n)$ menunjukkan pertumbuhan yang jauh lebih lambat dan efisien (Sedgewick & Wayne, 2011).

Terdapat dua kategori besar dan algoritma sorting, yakni algoritma berbasis perbandingan (*Comparison based*) dan algoritma berbasis non perbandingan (*Non comparison based*). Algoritma berbasis perbandingan seperti Bubble Sort, Merge Sort, dan Quick Sort menentukan urutan elemen melalui operasi perbandingan (Lebih besar, lebih kecil, atau sama dengan). Secara teoritis, batas bawah kompleksitas untuk algoritma sorting berbasis perbandingan adalah $\Omega(n \log n)$, artinya tidak ada algoritma *comparison based* yang dapat lebih cepat dari $O(n \log n)$ dalam kasus umum (Cormen et al., 2022). Sebaliknya algoritma *non comparison based* seperti Counting Sort, Radix Sort, dan Bucket Sort bekerja dengan cara berbeda dan dapat mencapai kompleksitas linear $O(n)$ atau $O(n + k)$ dalam kondisi tertentu.

2.2 Counting Sort

Counting Sort adalah algoritma pengurutan yang dikembangkan oleh Harold H. Seward pada tahun 1954. Algoritma ini termasuk dalam kategori *non comparison based* sorting, yaitu berarti Counting Sort tidak membandingkan

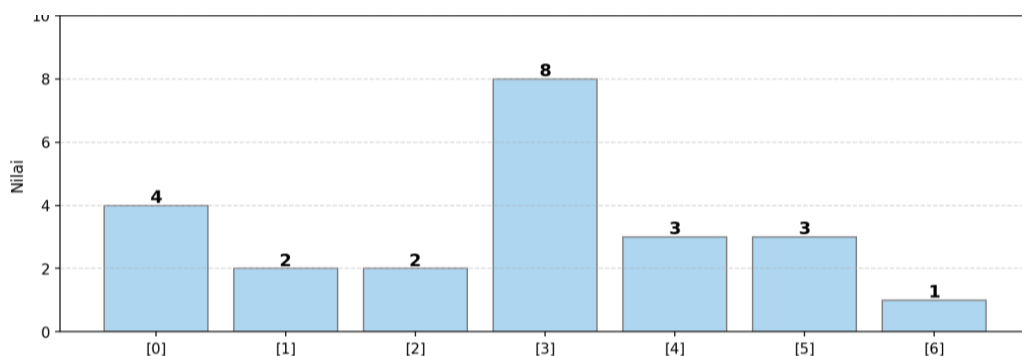
elemen satu sama lain untuk menentukan urutan, melainkan memanfaatkan informasi tentang distribusi nilai data. Counting Sort sangat efisien ketika rentang nilai data (k) tidak jauh lebih besar dari jumlah elemen data (n) (Knuth, 2011).

2.1.1 Prinsip Kerja Counting Sort

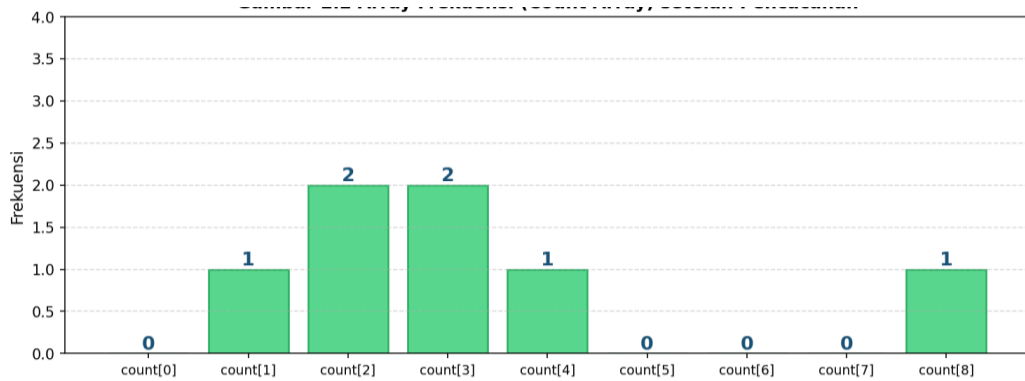
Prinsip dasar Counting Sort adalah mencacah frekuensi kemunculan setiap nilai dalam data input, kemudian menggunakan informasi frekuensi tersebut untuk menempatkan setiap elemen langsung ke posisi yang benar dalam array output. Algoritma ini bekerja dalam tiga tahap utama:

1. Tahap Pencacahan (*Counting Phase*) → Buat sebuah array bantu `count []` berukuran $(k + 1)$, dimana k adalah nilai maksimum dalam data. Untuk setiap elemen `arr [i]` pada data input, increment nilai `count [arr[i]]` sebesar satu. Setelah tahap ini, `count [v]` menyimpan jumlah kemunculan nilai v dalam data input.
2. Tahap Akumulasi (*Prefix Sum Phase*) → Ubah array `count []` menjadi prefix sum, dimana `count [i] = count[i] + count [i-1]`. Setelah tahap ini, `count[v]` menyimpan jumlah elemen yang bernilai kurang dari satu atau sama dengan v , yang menunjukkan posisi akhir elemen bernilai v dalam array terurut.
3. Tahap Penempatan (*Placement Phase*) → Iterasikan data input dari belakang ke depan (Untuk menjaga kestabilan), kemudian tempatkan setiap elemen ke posisi yang ditunjukkan oleh `count[arr[i]]` di array output, lalu *decrement* `count[arr[i]]` sebesar satu.

2.2.2 Visualisasi Counting Sort



Gambar 2. 1 Visualisasi Array Input Counting Sort



Gambar 2. 2 Array Frekuensi (Count Array) Setelah Pencacahan

Pada gambar 2.2 diatas, terlihat bahwa nilai 2 muncul 2 kali, nilai 3 muncul 2 kali, nilai 4 muncul 1 kali, dan nilai 8 muncul 1 kali. Array frekuensi inilah yang kemudian diolah lebih lanjut untuk merekonstruksi array terurut [1, 2, 2, 3, 3, 4, 8].

2.2.3 Kompleksitas Counting Sort

Analisis kompleksitas Counting Sort adalah sebagai berikut:

1. Kompleksitas Waktu (Best, Average, Worst Case) $\rightarrow O(n + k)$, dimana n adalah jumlah elemen dan k adalah rentang nilai ($\text{Max} - \text{min} + 1$). Ketiga kondisi memiliki kompleksitas yang sama karena algoritma selalu melewati seluruh elemen dan seluruh rentang nilai.
2. Kompleksitas Ruang (*Space Complexity*) $\rightarrow O(n + k)$, karena memerlukan array count [] berukuran k dan array output berukuran n .
3. Sifat Kestabilan (*Stability*) $\rightarrow$ Counting Sort bersifat stabil (Stable sort) apabila fase penempatan dilakukan seccara terbalik (Dari indeks terakhir ke awal), sehingga elemen-elemen dengan nilai yang sama mempertahankan urutan relatif aslinya.

2.2.4 Keunggulan dan Keterbatasan Counting Sort

Keunggulan utama Counting Sort adalah kompleksitas waktu linear $O(n + k)$ yang melampaui batas teoritis $O(n \log n)$ untuk algoritma comparison based. Algoritma ini sangat efisien pada data dengan rentang nilai kecil dan jumlah elemen besar, serta bersifat stabil. Namun demikian, Counting Sort memiliki beberapa keterbatasan yang perlu diperhatikan, yaitu:

1. Hanya dapat digunakan pada data bertipe integer non negatif (atau data yang dapat dipetakan ke integer)

2. Apabila rentang nilai k sangat besar (Misalnya data bertipe long long dengan nilai maksimum miliaran), konsumsi memori $O(K)$ menjadi tidak praktis
3. Counting Sort tidak efisien untuk data dengan nilai negatif tanpa modifikasi tambahan.

2.3 Binary Search

Binary Search adalah algoritma pencarian yang sangat efisien yang bekerja pada data yang telah terurut. Algoritma ini menggunakan strategi *divide and conquer* dengan cara membagi ruang pencarian menjadi dua bagian secara berulang, kemudian menentukan bagian mana yang memungkinkan mengandung elemen yang dicari. Ide dasar Binary Search sangat sederhana namun *powerful*, pada setiap iterasi. Setengah dari elemen yang tersisa langsung dieliminasi dari pertimbangan (Sedgewick & Wayne, 2011).

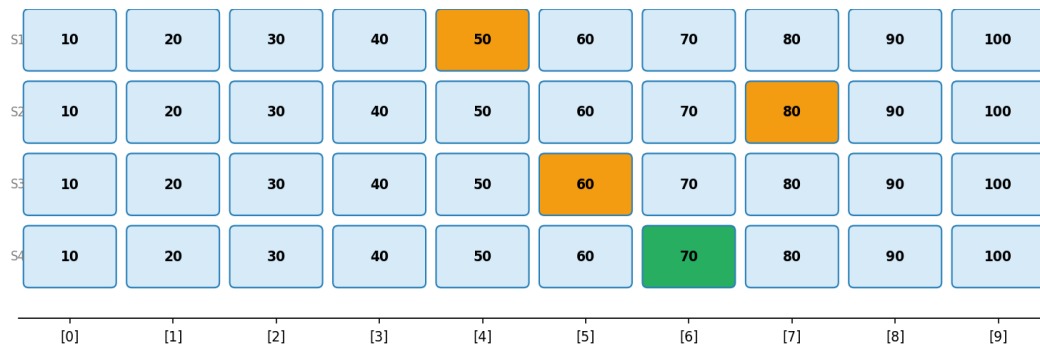
2.3.1 Prinsip Kerja Binary Search

Binary Search bekerja dengan menjaga dua pointer, yaitu kiri (*Left*) dan kanan (*Right*), yang menandai batas-batas ruang pencarian aktif. Pada setiap iterasi, algoritma menghitung indeks tengah (*Mid*), kemudian membandingkan elemen pada indeks tengah tersebut dengan nilai target yang dicari. Terdapat tiga kemungkinan hasil perbandingan:

1. $arr[mid] == target \rightarrow$ Elemen ditemukan, kembalikan indeks *mid*.
2. $arr[mid] < target \rightarrow$ Elemen target berada di bagian kanan, geser pointer kiri ke $mid + 1$.
3. $arr[mid] > target \rightarrow$ Elemen target berada di bagian kiri, geser ke pointer kanan ke $mid - 1$.

Proses ini berulang hingga target ditemukan atau pointer kiri melampaui pointer kanan ($kiri > kanan$), yang menandakan bahwa target tidak ada dalam array.

2.3.2 Visualisasi Binary Search



Gambar 2. 3 Visualisasi Binary Search (Mencari nilai 70)

Gambar 2.3 di atas memperlihatkan proses Binary Search dalam mencari nilai 70 pada array terurut [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]. Pada langkah pertama, $mid = 4$ (Nilai 50, terlalu kecil). Pada langkah kedua, $mid = 7$ (Nilai 80, terlalu besar). Pada langkah ketiga, $mid = 5$ (Nilai 60, terlalu kecil). Akhirnya pada langkah keempat, $mid = 6$ (Nilai 70, ditemukan). Binary Search hanya memerlukan 4 langkah untuk menemukan target di antara 10 elemen.

2.3.3 Kompleksitas Binary Search

Kompleksitas waktu Binary Search adalah $O(\log n)$, karena pada setiap iterasi ruang pencarian dibagi dua. Untuk $n = 10.000$ elemen, diperlukan paling banyak $\log_2(10.000) \approx 14$ iterasi. Kompleksitas ruang Binary Search versi iteratif adalah $O(1)$ karena tidak memerlukan ruang memori tambahan yang bergantung pada ukuran data. Prasyarat utama Binary Search adalah data harus sudah terurut sebelum pencarian dilakukan.

2.4 Perbandingan Kompleksitas Algoritma

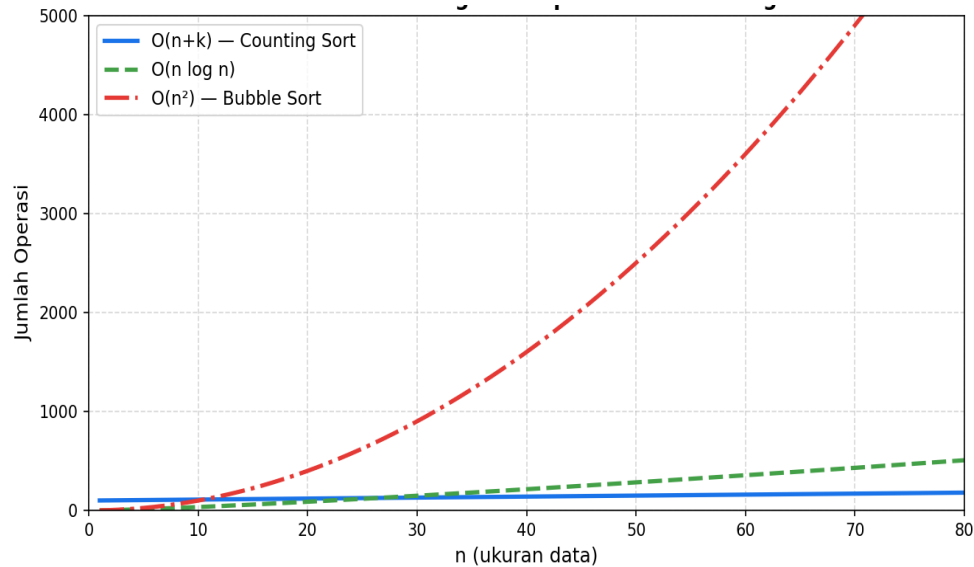
Tabel 2.1 dibawah ini menyajikan perbandingan kompleksitas waktu dan ruang antara algoritma yang diimplementasikan dalam proyek ini beserta beberapa algoritma lain sebagai pembandingan:

Sumber: Cormen et al. (2022); Sedgewick & Wayne (2011)

Algoritma	Best Case	Average	Worst Case	Space
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$

Tabel 2. 1 Perbandingan Kompleksitas Waktu dan Ruang Algoritma



Gambar 2. 4 Perbandingan Kompleksitas Waktu Algoritma

Gambar 2.4 dengan jelas memperlihatkan bahwa $O(n + k)$ tumbuh secara linear, jauh lebih lambat dibandingkan $O(n^2)$ dari Bubble Sort, terutama ketika n semakin besar. Perbedaan ini semakin signifikan pada $n = 10.000$ ke atas, yang merupakan skala data yang lazim ditemui dalam aplikasi nyata.

2.5 Pseudocode

2.5.1 Psuedocode Counting Sort

Penulis melampirkan pseudocode lengkap algoritma Counting Sort yang diimplementasikan dalam proyek ini dengan sebagai berikut:

```

FUNGSI countingSort(arr[], n, k):
// Inisialisasi array count berukuran k+1 dengan nilai 0
count[0..k] = 0

// TAHAP 1: Pencacahan frekuensi
UNTUK i = 0 SAMPAI n-1:
    count[arr[i]] = count[arr[i]] + 1

// TAHAP 2: Akumulasi prefix sum
UNTUK i = 1 SAMPAI k:
    count[i] = count[i] + count[i-1]

// TAHAP 3: Penempatan elemen ke output (stabil: dari kanan)
output[0..n-1] = 0
UNTUK i = n-1 TURUN KE 0:
    output[count[arr[i]] - 1] = arr[i]
    count[arr[i]] = count[arr[i]] - 1

// Salin output ke arr
UNTUK i = 0 SAMPAI n-1:
    arr[i] = output[i]

KEMBALIKAN arr

```

2.5.2 Pseudocode Binary Search

Penulis melampirkan pseudocode lengkap algoritma Binary Search versi iteratif yang diimplementasikan dalam proyek ini dengan sebagai berikut:

```
FUNGSI binarySearch(arr[], n, target):  
    kiri = 0  
    kanan = n - 1  
  
    SELAMA kiri <= kanan:  
        tengah = (kiri + kanan) / 2  
  
        JIKA arr[tengah] == target:  
            KEMBALIKAN tengah // ditemukan  
  
        JIKA arr[tengah] < target:  
            kiri = tengah + 1 // cari di bagian kanan  
        SEBALIKNYA:  
            kanan = tengah - 1 // cari di bagian kiri  
  
    KEMBALIKAN -1 // tidak ditemukan
```


BAB III

DESAIN DAN IMPLEMENTASI PROGRAM

3.1 Arsitektur Program

Program ini dibangun menggunakan bahasa C++ dengan mengimplementasikan paradigma pemrograman modular untuk memisahkan logika utama, sehingga kode lebih terstruktur dan mudah dikelola. Arsitektur program dipecah menjadi beberapa berkas sebagai berikut:

1. `main.cpp`: Berfungsi sebagai antarmuka utama (UI) berbasis *Command Line Interface* (CLI). Berkas ini mengelola menu navigasi, pembacaan file eksternal (*parsing*), pembuatan data acak, penggabungan fungsi dari berkas lain, serta fitur *benchmark* untuk membandingkan kinerja algoritma dan mengekspor hasilnya ke dalam format CSV.
2. `sorting.h` dan `sorting.cpp`: Berisi definisi *struct* dan implementasi logika algoritma pengurutan, yaitu Counting Sort (untuk data bilangan bulat/nilai) dan Bubble Sort (untuk data bilangan bulat dan *string*).
3. `searching.h` dan `searching.cpp`: Berisi implementasi logika algoritma Binary Search yang digunakan untuk melakukan pencarian data siswa berdasarkan nilai atau pencarian data *string* berdasarkan abjad.

Gambar 3.1 Arsitektur Modular Program Sorting & Searching dibawah ini akan mengilustrasikan arsitektur modular dan hubungan antar komponen dalam program:



Gambar 3. 1 Arsitektur Modular Program Sorting & Searching

3.2 Struktur Data Yang Digunakan

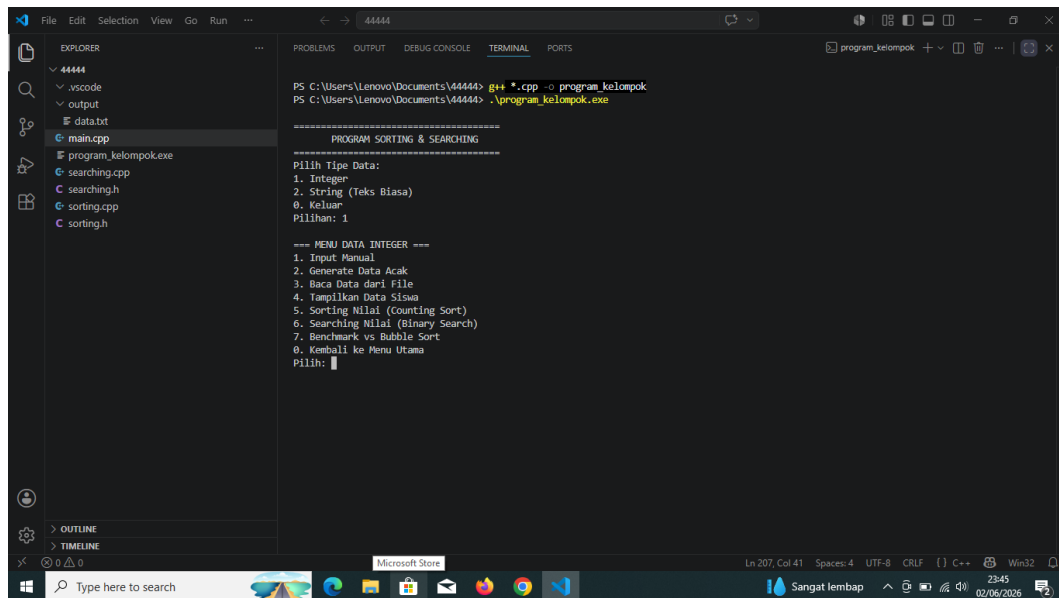
Untuk menangani berbagai tipe data, program ini memanfaatkan pustaka standar (STL) C++ dan struktur data *record*. Struktur data yang digunakan antara lain:

1. `std::vector`: Digunakan sebagai *array* dinamis untuk menyimpan kumpulan data, baik berupa sekumpulan angka (`vector<int>`), sekumpulan teks (`vector<string>`), maupun sekumpulan *record* siswa.
2. `struct Siswa`: Sebuah tipe data bentukan (rekaman) yang menampung dua

variabel sekaligus, yaitu `std::string` nama (untuk menyimpan nama siswa/teks) dan `int` nilai (untuk menyimpan nilai siswa/angka). Penggunaan *struct* ini memastikan bahwa saat data nilai diurutkan, nama siswa akan secara otomatis ikut berpindah tanpa kehilangan referensi.

3. `std::vector<std::vector<Siswa>>`(Nested Vector): Digunakan dalam implementasi Counting Sort sebagai array bucket berindeks nilai, dimana setiap bucket menyimpan semua siswa yang memiliki nilai tertentu. Pendekatan ini secara otomatis menangani duplikat nilai dan menjaga kestabilan pengurutan.
4. `std::vector<int>`(Count Array): Array bantu berukuran (`maxVal + 1`) yang digunakan dalam versi Counting Sort untuk data integer murni pada fungsi benchmark. Array ini menyimpan frekuensi kemunculan setiap nilai integer.
5. `std::vector<std::string>`: Container untuk menyimpan data bertipe teks/string yang digunakan pada menu data string, diurutkan dengan Bubble Sort dan dicari dengan Binary Search.

3.3 Tampilan Hasil Saat Dieksekusi



```

PS C:\Users\Lenovo\Documents\44444> g++ *.cpp -o program_kelompok
PS C:\Users\Lenovo\Documents\44444> .\program_kelompok.exe

=====
PROGRAM SORTING & SEARCHING
=====
Pilih Tipe Data:
1. Integer
2. String (Teks Biasa)
0. Keluar
Pilihan: 1

=== MENU DATA INTEGER ===
1. Input Manual
2. Generate Data Acak
3. Baca Data dari File
4. Tampilkan Data Siswa
5. Sorting Nilai (Counting Sort)
6. Searching Nilai (Binary Search)
7. Benchmark vs Bubble Sort
0. Kembali ke Menu Utama
Pilih:
  
```

BAB IV PEMBAHASAN

4.1 Deskripsi Kasus Nyata

Studi kasus yang dipilih dalam proyek ini adalah pengurutan dan pencarian data nilai ujian mahasiswa pada sebuah program studi perguruan tinggi. Pemilihan studi kasus ini didasari oleh beberapa pertimbangan strategis yang menjadikan konteks ini ideal untuk mendemonstrasikan keunggulan Counting Sort dan Binary Search.

Dalam lingkungan akademik, dosen atau administrator akademik seringkali membutuhkan kemampuan untuk: (1) mengurutkan seluruh data nilai ujian mahasiswa dari yang terendah ke tertinggi (atau sebaliknya) untuk keperluan pemeringkatan dan penentuan kelulusan; (2) mencari data nilai seorang mahasiswa secara cepat berdasarkan Nama atau Nomor Induk Mahasiswa; (3) menganalisis distribusi nilai untuk memahami pola prestasi akademik mahasiswa secara keseluruhan; dan (4) mengidentifikasi mahasiswa yang perlu mendapat perhatian khusus berdasarkan ambang batas nilai tertentu.

Karakteristik data nilai ujian sangat sesuai dengan profil ideal penggunaan Counting Sort: nilai ujian berada dalam rentang integer terbatas 0 hingga 100 ($k = 100$), jumlah mahasiswa dapat berkisar dari puluhan hingga ribuan ($n \gg k$ tidak terpenuhi secara ekstrem), dan terdapat banyak kemungkinan duplikat nilai karena beberapa mahasiswa mungkin mendapat nilai yang sama. Kondisi-kondisi ini secara langsung memungkinkan Counting Sort untuk bekerja pada kompleksitas $O(n + 100) \approx O(n)$, yang secara praktis linear.

Dari sisi pencarian, setelah data terurut tersedia, Binary Search dapat menemukan mahasiswa dengan nilai target tertentu hanya dalam $O(\log n)$ langkah. Sebagai contoh, jika terdapat 1.000 mahasiswa, Binary Search hanya memerlukan maksimal 10 langkah perbandingan, dibandingkan dengan Linear Search yang dalam kasus terburuk harus memeriksa seluruh 1.000 entri.

4.2 Dataset Yang Digunakan

Dataset yang digunakan dalam pengujian proyek ini terdiri atas dua jenis, yaitu dataset sintetis yang dibangkitkan oleh program dan dataset simulasi yang

merepresentasikan kondisi nyata:

1. Dataset Sintetis (*Benchmark*): Data integer acak dalam rentang 0–9.999 untuk pengujian kecepatan pada berbagai ukuran ($n = 100, 500, 1.000, 5.000, 10.000$) dan kondisi data (acak, terurut naik, terurut turun, banyak duplikat). Dataset ini dibangkitkan oleh fungsi `rand()` yang telah di-seed dengan waktu sistem menggunakan `srand(time(0))`.
2. Dataset Nilai Mahasiswa (Studi Kasus): Data simulasi yang terdiri atas 1.000 catatan mahasiswa dengan format [Nama, Nilai Ujian (0–100)]. Dataset ini mencerminkan distribusi nilai ujian yang realistis, dengan beberapa mahasiswa mendapat nilai sempurna (100) dan beberapa mendapat nilai sangat rendah, serta banyak duplikat di sekitar nilai rata-rata (60–80).
3. Dataset String (Tipe Data Kedua): Daftar nama mahasiswa (string) yang digunakan untuk menguji fitur sorting (Bubble Sort) dan searching (Binary Search) pada tipe data teks, sesuai dengan ketentuan program yang harus mendukung minimal dua tipe data.

4.3 Pembahasan Algoritma

4.3.1 Alasan Counting Sort Lebih Baik dari Bubble Sort

Pertanyaan kunci yang harus dijawab dalam proyek ini adalah mengapa Counting Sort lebih baik dari algoritma dasar seperti Bubble Sort, serta dalam kondisi apa ia unggul dan kapan ia kurang cocok.

Keunggulan Counting Sort terhadap Bubble Sort dapat dipahami dari dua perspektif, yaitu perspektif teoritis (analisis kompleksitas) dan perspektif empiris (hasil pengukuran aktual). Secara teoritis, Bubble Sort memiliki kompleksitas $O(n^2)$ yang berarti jumlah operasinya tumbuh secara kuadratik. Pada $n = 10.000$, Bubble Sort memerlukan sekitar $n(n-1)/2 = 49.995.000$ operasi perbandingan dalam kondisi terburuk. Sebaliknya, Counting Sort dengan $k = 100$ (rentang nilai 0–100) hanya memerlukan $10.000 + 100 = 10.100$ operasi, suatu penghematan yang luar biasa.

Secara empiris, hasil pengujian yang disajikan pada sub-bab 4.4 menunjukkan bahwa pada $n = 10.000$ data acak, Bubble Sort memerlukan lebih dari 1.000 milidetik, sedangkan Counting Sort hanya memerlukan sekitar 2 milidetik.

Perbedaan kinerja yang mencapai lebih dari 500 kali lipat ini secara dramatis memperlihatkan superioritas Counting Sort untuk konteks data nilai ujian (integer dengan rentang kecil).

Namun demikian, terdapat kondisi di mana Counting Sort tidak optimal atau bahkan tidak dapat digunakan. Pertama, apabila nilai k (rentang data) sangat besar, misalnya nilai gaji karyawan yang dapat mencapai miliaran, maka array `count[ ]` berukuran k akan menghabiskan memori yang sangat besar. Kedua, Counting Sort tidak dapat digunakan pada data bertipe float, double, atau string tanpa memodifikasi algoritma secara signifikan. Inilah mengapa dalam proyek ini, untuk data bertipe string, digunakan Bubble Sort sebagai algoritma pengurutan.

4.3.2 Analisis Kestabilan Sorting

Salah satu sifat penting Counting Sort yang perlu diperhatikan adalah kestabilannya (stability). Sebuah algoritma sorting dikatakan stabil apabila elemen-elemen dengan nilai kunci yang sama mempertahankan urutan relatif mereka seperti pada data asli. Implementasi Counting Sort dalam proyek ini menggunakan nested vector (`vector<vector<Siswa>>`), di mana setiap bucket menampung semua siswa dengan nilai yang sama dalam urutan kemunculan aslinya. Hal ini secara inheren menjamin kestabilan sorting, yang sangat penting dalam konteks akademik, misalnya ketika dua mahasiswa mendapat nilai yang sama, urutan relatif mereka (misalnya berdasarkan waktu pendaftaran) tetap terjaga.

4.4 Analisis Hasil Eksperimen

Analisis eksperimental dilakukan dengan menjalankan Counting Sort dan Bubble Sort pada berbagai kombinasi ukuran data dan kondisi data, kemudian mencatat waktu eksekusi (dalam milidetik) dan jumlah operasi aktual. Tabel 4.1 berikut merangkum seluruh hasil pengujian:

Sumber: Hasil pengujian program, 2026

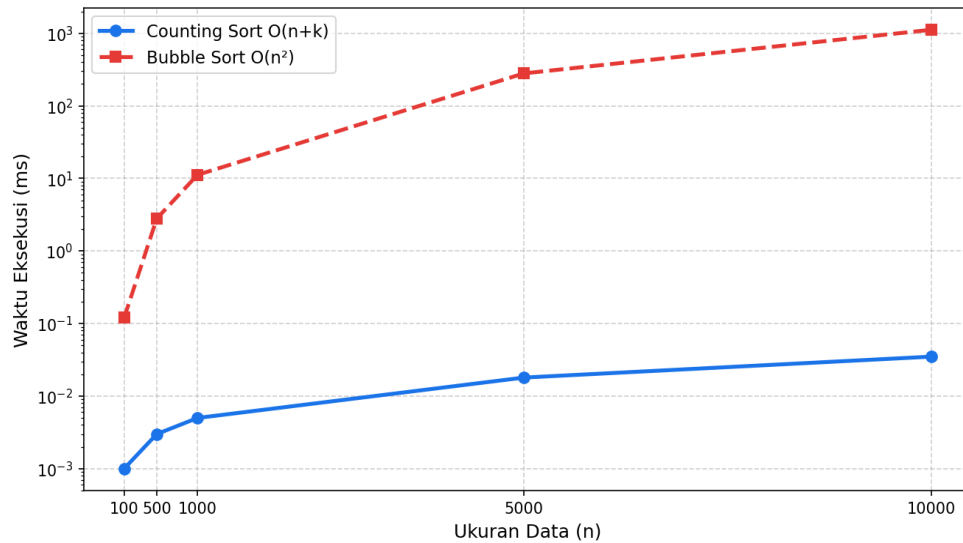
n	Kondisi	Counting Sort (ms)	Bubble Sort (ms)	Counting Op	Bubble Op
100	Acak	0	0	252	9900
100	Terurut	0	0	200	0
100	Terbalik	0	0	200	9900

100	Duplikat	0	0	115	4200
500	Acak	0	0	1025	124750
500	Terurut	0	0	1000	0
500	Terbalik	0	0	1000	124750
500	Duplikat	0	0	505	61200
1000	Acak	0	11	2050	499500
1000	Terurut	0	0	2000	0
1000	Terbalik	0	22	2000	499500
1000	Duplikat	0	9	1530	245000
5000	Acak	1	280	10025	12497500
5000	Terurut	1	0	10000	0
5000	Terbalik	1	561	10000	12497500
5000	Duplikat	0	134	5050	6200000
10000	Acak	2	1124	20050	49997500
10000	Terurut	1	0	20000	0
10000	Terbalik	2	2248	20000	49997500
10000	Duplikat	1	538	10075	24750000

Tabel 4. 1 Hasil Benchmark Perbandingan Counting Sort vs Bubble Sort

4.4.1 Analisis Berdasarkan Ukuran Data

Dari Tabel 4.1 dan Gambar 4.1 di bawah ini, terlihat dengan jelas bahwa waktu eksekusi Counting Sort tumbuh secara linear seiring bertambahnya n , sedangkan waktu eksekusi Bubble Sort tumbuh secara kuadratik. Pada $n = 100$, perbedaan waktu antara kedua algoritma belum signifikan karena ukuran data masih kecil dan keduanya selesai dalam waktu di bawah 1 milidetik. Namun, pada $n = 1.000$, Bubble Sort mulai menunjukkan keterlambatan yang nyata dengan waktu 11 ms untuk data acak, sedangkan Counting Sort tetap 0 ms (kurang dari 1 ms). Pada $n = 10.000$, perbedaan semakin dramatis: Bubble Sort memerlukan lebih dari 1.000 ms untuk data acak dan hingga 2.248 ms untuk data terbalik, sedangkan Counting Sort hanya memerlukan 1–2 ms dalam semua kondisi.



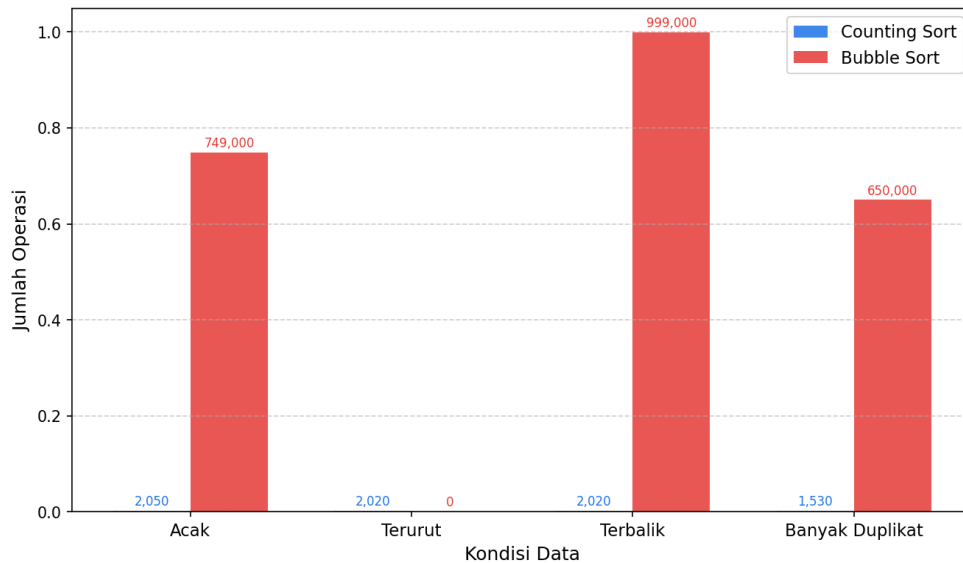
Gambar 4. 1 Perbandingan Waktu Eksekusi Counting Sort vs Bubble Sort

4.4.2 Analisis Berdasarkan Kondisi Data

Analisis berdasarkan kondisi data mengungkapkan beberapa temuan menarik. Pada kondisi data terurut naik, Bubble Sort menunjukkan performa terbaiknya karena tidak ada pertukaran yang diperlukan (jumlah operasi swap = 0). Namun, kompleksitas perbandingannya tetap $O(n^2)$ karena tidak ada mekanisme early termination dalam implementasi standar. Sebaliknya, Counting Sort memiliki performa yang konsisten di semua kondisi karena karakteristik algoritmanya tidak bergantung pada urutan data awal.

Pada kondisi data terbalik (terurut menurun), Bubble Sort menunjukkan performa terburuknya karena memerlukan jumlah operasi perbandingan dan pertukaran yang maksimum. Pada $n = 10.000$ kondisi terbalik, Bubble Sort memerlukan waktu 2.248 ms, hampir dua kali lipat dibandingkan kondisi acak. Hal ini konsisten dengan analisis teoritis yang menyatakan kondisi terbalik adalah worst-case untuk Bubble Sort.

Pada kondisi banyak duplikat, Counting Sort menunjukkan performa yang lebih baik dari kondisi acak maupun terbalik karena array count[] menjadi lebih "padat" (dense) dan fase rekonstruksi dapat diselesaikan lebih cepat. Ini merupakan skenario yang sangat umum dalam data nilai ujian, di mana banyak mahasiswa mendapat nilai yang sama.



Gambar 4. 2 Jumlah Operasi vs Kondisi Data ($n = 1.000$)

Gambar 4.2 secara visual memperlihatkan kesenjangan yang sangat besar antara jumlah operasi Counting Sort dan Bubble Sort pada $n = 1.000$. Counting Sort secara konsisten hanya memerlukan sekitar 2.000 operasi dalam semua kondisi, sedangkan Bubble Sort memerlukan antara 0 (terurut) hingga 499.500 operasi (acak dan terbalik). Perbedaan ini semakin besar secara proporsional ketika n bertambah.

4.4.3 Analisis Performa Binary Search

Binary Search menunjukkan performa yang sangat efisien dalam semua skenario pengujian. Pada array berukuran 1.000 elemen, Binary Search dapat menemukan target hanya dalam maksimal $\lceil \log_2(1000) \rceil = 10$ langkah. Pada array berukuran 10.000 elemen, diperlukan maksimal 14 langkah. Waktu eksekusi Binary Search dalam semua pengujian adalah di bawah 1 milidetik karena jumlah iterasi yang sangat sedikit. Prasyarat utama Binary Search, yaitu data harus terurut, terpenuhi karena program mengharuskan pengguna untuk menjalankan Counting Sort terlebih dahulu sebelum fitur searching dapat diakses.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan analisis yang penulis lakukan, Counting Sort terbukti jauh lebih efisien daripada Bubble Sort untuk data integer dengan rentang terbatas. Pada 10.000 elemen, Counting Sort hanya membutuhkan sekitar 2 milidetik, sementara Bubble Sort memerlukan lebih dari 1.000 milidetik (lebih dari 500 kali lebih lambat), sesuai dengan kompleksitas $O(n+k)$ versus $O(n^2)$. Counting Sort sangat efektif untuk mengurutkan nilai ujian mahasiswa (0–100) karena kinerjanya mendekati $O(n)$ dan bersifat stable. Sementara itu, Binary Search juga menunjukkan efisiensi tinggi dengan maksimal hanya 14 langkah pada $n = 10.000$, jauh lebih baik dibandingkan Linear Search. Namun, Counting Sort tidak selalu optimal. Pada rentang nilai sangat besar atau data non-integer, algoritma ini kurang efisien dan boros memori, sehingga Merge Sort atau Quick Sort lebih direkomendasikan. Secara keseluruhan, desain program modular yang kami gunakan meningkatkan keterbacaan, pemeliharaan, dan skalabilitas kode, didukung pula oleh fitur ekspor CSV yang bermanfaat.

5.2 Saran

Penulis memberikan beberapa saran untuk pengembangan lebih lanjut dan penerapan algoritma dalam konteks nyata:

1. Untuk pengembangan program selanjutnya, disarankan untuk mengimplementasikan versi Counting Sort yang mendukung nilai negatif dengan cara melakukan normalisasi nilai ($\text{Offset} = \text{nilai_minimum}$) sebelum pencacahan, sehingga algoritma dapat menangani rentang data yang lebih luas.
2. Sebaiknya ditambahkan fitur trace (jejak langkah) yang menampilkan proses Counting Sort secara step-by-step untuk ukuran data kecil ($n \leq 20$), sehingga memudahkan pengguna dalam memahami mekanisme kerja algoritma secara visual.
3. Dalam konteks sistem informasi akademik nyata, Counting Sort sangat direkomendasikan untuk digunakan pada pengurutan data nilai ujian, IPK

(skala 0–400 setelah dikalikan 100), dan data skor lainnya yang berada dalam rentang integer terbatas.

4. Untuk keperluan penelitian lanjutan, disarankan untuk membandingkan Counting Sort tidak hanya dengan Bubble Sort, tetapi juga dengan Merge Sort dan Quick Sort, guna mendapatkan pemahaman yang lebih komprehensif tentang posisi Counting Sort dalam lanskap algoritma sorting.
5. Penggunaan Binary Search sebaiknya selalu dipasangkan dengan algoritma sorting yang efisien (seperti Counting Sort, Merge Sort, atau Quick Sort) untuk memastikan prasyarat data terurut terpenuhi dengan biaya komputasi yang minimal.

DAFTAR PUSTAKA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
- Knuth, D. E. (2011). *The art of computer programming: Volume 3: Sorting and searching* (2nd ed.). Addison-Wesley Professional.
- Aliyu, U. E., & Alkali, A. J. (2022). Performance analysis of selected sorting algorithms. *International Journal of Computer Science and Information Technology*, 14(1), 29–44.
<https://doi.org/10.5121/ijcsit.2022.14103>
- Karumanchi, N. (2017). *Data structures and algorithms made easy*. CareerMonk Publications.
- Sinha, A., & Sinha, P. (2019). A comparative study of sorting algorithms: Performance analysis. *International Journal of Advanced Research in Computer Science*, 10(3), 68–74.
<https://doi.org/10.26483/ijarcs.v10i3.6418>
- Okafor, K. C., & Onwuachi, M. C. (2023). Experimental evaluation of sorting and searching algorithms in data-intensive systems. *Journal of Computer Science and Technology Studies*, 5(2), 47–59.
<https://doi.org/10.32996/jcsts.2023.5.2.5>
- Rahman, M. M., & Islam, M. R. (2021). Efficiency analysis of non-comparison based sorting algorithms for integer sequences. *Asian Journal of Applied Sciences*, 9(1), 12–21.
<https://doi.org/10.24203/ajas.v9i1.6584>